Apuntes de PHP

Referencia rápida: de las variables a la programación orientada a objetos

Programación PHP · Curso 2026-2027

Índice

1. Fundamentos — Variables, tipos, operadores, constantes

2. Estructuras de control — if, switch, bucles

3. Arrays — Tipos, funciones, ordenar, técnicas habituales

4. Funciones — Propias, texto, números y formato

5. PHP y HTML — Sintaxis alternativa, atributos, seguridad

6. Organizar el proyecto — include, require, partials

7. Formularios, GET y POST — Validación, redirecciones, controllers

8. Sesiones — Mensajes flash, proteger páginas

9. Programación orientada a objetos — Clases, herencia, colecciones

10. Errores habituales y cómo depurar — Mensajes de error, lista de comprobación

Chuleta final — Referencia rápida

Cada tema incluye explicación, dos ejemplos y, cuando hace falta, avisos con los errores más comunes. Los bloques en rojo (**Atención**) y verde (**Consejo**) resumen lo que más se suele fallar.

1. Fundamentos

Estructura de un archivo PHP

El código PHP va entre `<?php` y `?>`. Todo lo que queda fuera de esas etiquetas se envía al navegador tal cual (normalmente HTML). Cada instrucción termina en `;`. Si el archivo solo contiene PHP, no se escribe el `?>` final.

```
<?php
echo "Hola mundo";      // echo muestra un texto
// Comentario de una línea
# También es un comentario de una línea
/* Comentario
   de varias líneas */
```

Ejemplo 1 — PHP mezclado con HTML:

```
<h1>Mi página</h1>
<p>Hoy es <?php echo date("d/m/Y"); ?></p>
<p>Dos más dos son <?= 2 + 2 ?></p>
```

Ejemplo 2 — echo, concatenación y saltos de línea:

```
echo "Hola ", "mundo";            // echo admite varios valores separados por
coma
echo "Total: " . 15 . " euros";   // el punto concatena (une) textos
echo "Línea 1<br>Línea 2";        // en el navegador el salto de línea es <br>
echo "Línea 1\nLínea 2";          // \n NO se ve en el navegador, solo en el
código fuente
```

Atención: Olvidar el `;` al final de una línea es el error más habitual. PHP muestra «Parse error: syntax error, unexpected…» y suele señalar la línea siguiente a la que falta.

```
// MAL: Falta el ; en la primera línea
echo "Hola"
echo "Adiós";

// BIEN: Correcto
echo "Hola";
echo "Adiós";
```

Variables

Una variable empieza por `$`, no necesita declarar su tipo y distingue mayúsculas de minúsculas (`$nombre` y `$Nombre` son variables distintas). Puede contener letras, números y `_`, pero no puede empezar por un número.

```
$nombreVariable = valor;
```

Ejemplo 1 — Crear y mostrar variables:

```
$nombre  = "Lucía";
$edad    = 17;
$altura  = 1.65;
$esSocio = true;

echo "$nombre tiene $edad años";   // → Lucía tiene 17 años
```

Ejemplo 2 — Variables y arrays dentro de un texto:

```
$libro = ["titulo" => "Dune", "autor" => "Herbert"];
$paginas = 612;

echo "Título: {$libro['titulo']}";       // con llaves dentro de comillas dobles
echo 'Título: ' . $libro["titulo"];      // o concatenando con el punto
echo "Tiene $paginas páginas";           // → Tiene 612 páginas
echo 'Tiene $paginas páginas';           // → Tiene $paginas páginas (no se
sustituye)
```

Atención: Las comillas simples `'...'` no sustituyen variables ni interpretan `\n`. Las comillas dobles `"..."` sí. Si dudas, concatena con el punto.

Tipos de datos

Tipo	Ejemplo	Descripción
string	`"Hola"`	Texto
int	`42` `-7`	Número entero
float	`3.14` `0.5`	Número con decimales (se escribe con punto)
bool	`true` `false`	Verdadero o falso
array	`[1, 2, 3]`	Lista de valores
null	`null`	Ausencia de valor

Ejemplo 1 — Comprobar el tipo con var_dump() y gettype():

```
var_dump(42);         // → int(42)
var_dump("42");       // → string(2) "42"
var_dump(3.5);        // → float(3.5)
var_dump(true);       // → bool(true)
echo gettype(5.5);    // → double (así llama PHP al float)
```

Ejemplo 2 — Conversión entre tipos:

```
$a = "5" + 3;          // → 8   (PHP convierte el texto a número)
$b = (int) "12abc";    // → 12
$c = (float) "3.5";    // → 3.5
$d = (string) 25;      // → "25"
$e = intval("7.9");    // → 7   (quita los decimales, no redondea)

var_dump(is_numeric("12.5"));  // → true
var_dump(is_numeric("abc"));   // → false
```

Atención: La división `/` devuelve decimales cuando no es exacta: `10 / 4` → 2.5. Para la división entera usa `intdiv(10, 4)` → 2.

Atención: Al mostrar un booleano, `echo true` imprime `1` y `echo false` no imprime nada. Para ver su valor real usa `var_dump()` o un ternario (`$x ? "Sí" : "No"`).

Operadores

Tipo	Operadores	Ejemplo
Aritméticos	`+` `-` `*` `/` `%` `**`	`17 % 5` → 2 (resto) `2 ** 3` → 8 (potencia)
Asignación	`=` `+=` `-=` `*=` `/=` `.=`	`$x += 3` equivale a `$x = $x + 3`

Tipo	Operadores	Ejemplo
Incremento	`++ --`	`$i++` suma 1 a `$i`
Comparación	`== === != !== < > <= >=`	`$edad >= 18`
Lógicos	`&& || !`	`$a > 0 && $b > 0`
Concatenación	`. .=`	`"Hola " . $nombre`

Ejemplo 1 — Aritmética y asignación compuesta:

```
$x = 17;
echo $x % 5;          // → 2   (resto de la división)
echo $x ** 2;         // → 289 (potencia)
echo intdiv($x, 5);   // → 3   (división entera)

$x += 3;              // → 20
$texto = "Hola";
$texto .= " mundo";   // → "Hola mundo"
```

Ejemplo 2 — Comparar con == y con ===:

```
var_dump(5 == "5");        // → true   (compara solo el valor)
var_dump(5 === "5");       // → false  (compara valor Y tipo)
var_dump(null == false);   // → true
var_dump(null === false);  // → false

// Los datos de un formulario llegan siempre como texto:
$id = "3";
var_dump($id === 3);       // → false
var_dump((int) $id === 3); // → true
```

Consejo: Usa siempre `===` y `!==`, salvo que tengas un motivo para ignorar el tipo.

Atención: Los números decimales no son exactos: `0.1 + 0.2 === 0.3` da `false`. Si necesitas comparar decimales o evitar errores acumulados, redondea con `round($n, 2)`.

Operador ternario y operador ??

El **ternario** es un `if/else` en una línea. El operador **`??`** devuelve el valor de la izquierda si existe y no es `null`; si no, el de la derecha. Se usa muchísimo con `$_GET`, `$_POST` y claves de arrays que pueden no existir.

```
condición ? valorSiTrue : valorSiFalse
$variable ?? valorPorDefecto
```

Ejemplo 1 — Ternario:

```
$edad = 20;
$tipo = $edad >= 18 ? "adulto" : "menor";
echo "Eres $tipo";                      // → Eres adulto

$stock = 0;
echo $stock > 0 ? "Disponible" : "Agotado";
```

Ejemplo 2 — ?? para valores que pueden no existir:

```
$q = $_GET["q"] ?? "";                    // si no hay "q" en la URL, usa ""

$visitas = [];
$visitas["lunes"] = ($visitas["lunes"] ?? 0) + 1;   // si no existía, parte de 0
echo $visitas["lunes"];                   // → 1
```

Constantes

Una constante guarda un valor que **no cambia** durante toda la ejecución. Se escribe sin `$` y, por convención, en mayúsculas.

	define()	const
Sintaxis	`define("IVA", 21);`	`const IVA = 21;`
Dónde se puede usar	En cualquier sitio (también dentro de un `if`)	Al principio del archivo o dentro de clases
Valor	Cualquier tipo, incluidos arrays	Cualquier tipo, incluidos arrays

Ejemplo 1 — Declarar y usar constantes:

```
define("IVA", 21);
const NOMBRE_TIENDA = "Mi tienda";

echo IVA;                      // → 21   (sin $)
echo NOMBRE_TIENDA;

define("ROLES", ["ADMIN", "USER"]);
echo ROLES[0];                 // → ADMIN
```

Ejemplo 2 — Constantes en un archivo de configuración:

```
// config.php
define("DESCUENTO_MAX", 30);

// otro.php
require "config.php";

function limitar($descuento) {
    // las constantes SÍ se ven dentro de las funciones
    return $descuento > DESCUENTO_MAX ? DESCUENTO_MAX : $descuento;
}

echo limitar(45);              // → 30
```

Atención: Una constante no se puede modificar: `define("IVA", 10);` una segunda vez da un aviso y no cambia su valor.

2. Estructuras de control

if / elseif / else

PHP evalúa las condiciones **de arriba abajo** y ejecuta solo el primer bloque cuyo resultado sea verdadero. Si ninguno lo es, se ejecuta el `else`.

```
if (condición1) {
    // se ejecuta si condición1 es verdadera
} elseif (condición2) {
    // se ejecuta si condición1 es falsa y condición2 verdadera
} else {
    // se ejecuta si ninguna anterior se cumple
}
```

Ejemplo 1 — Tres tramos de precio:

```
$edad = 70;
$esSocio = false;

if ($edad < 5) {
    $precio = 0;
} elseif ($edad >= 65 || $esSocio) {
    $precio = 5;
} else {
    $precio = 10;
}
echo "Entrada: $precio €";     // → Entrada: 5 €
```

Ejemplo 2 — Contar cuántas condiciones se cumplen y decidir con el contador:

```
$s1 = 12; $s2 = 8; $s3 = 15; $s4 = 3;     // lecturas de cuatro sensores
$minimo = 10;

$fallos = 0;
if ($s1 < $minimo) { $fallos++; }
if ($s2 < $minimo) { $fallos++; }
if ($s3 < $minimo) { $fallos++; }
if ($s4 < $minimo) { $fallos++; }

if ($fallos === 0) {
    echo "Todo correcto";
} elseif ($fallos <= 2 && $s4 >= 5) {
    echo "Revisión recomendada";
} else {
    echo "Avería";                      // → Avería (2 fallos pero $s4 < 5)
}
```

Consejo: Cuando una decisión depende de «todos», «ninguno» o «alguno», lo más claro es contar con una variable (`$fallos`) y decidir al final, en lugar de escribir una condición enorme.

Operador	Significado	Ejemplo
`&&`	Y: se cumplen las dos	`$a >= 5 && $b >= 5`
`||`	O: se cumple al menos una	`$a < 4 || $b < 4`
`!`	No: invierte el resultado	`!$esSocio`

Atención: Cuando mezcles `&&` y `||` usa paréntesis. `$a || $b && $c` no significa lo mismo que `($a || $b) && $c`, porque `&&` se evalúa antes que `||`.

switch

Compara una variable con varios valores posibles. Cada `case` debe terminar con `break`, y `default` recoge los demás casos.

```
switch ($variable) {
    case valor1:
        // ...
        break;
    case valor2:
        // ...
        break;
    default:
        // ninguno coincide
}
```

Ejemplo 1 — Un valor por case:

```
$dia = "sábado";

switch ($dia) {
    case "sábado":
    case "domingo":
        echo "Fin de semana";
        break;
    default:
        echo "Entre semana";
}
// → Fin de semana
```

Ejemplo 2 — Agrupar valores y guardar el resultado en una variable:

```
$mes = 2;

switch ($mes) {
    case 12:
    case 1:
    case 2:
        $estacion = "invierno";
        break;
    case 3:
    case 4:
    case 5:
        $estacion = "primavera";
        break;
    default:
        $estacion = "otra";
}
echo $estacion;      // → invierno
```

Atención: Si te olvidas del `break`, PHP sigue ejecutando el `case` siguiente. Solo se omite a propósito cuando quieres agrupar varios valores, como en el ejemplo anterior. Además, `switch` compara con `==`, no con `===`.

Bucle for

Se usa cuando sabes cuántas veces repetir. Tiene tres partes: inicio, condición para seguir y paso.

```
for ($i = inicio; $i <= fin; $i++) {
    // se repite mientras la condición sea verdadera
}
```

Ejemplo 1 — Cuenta atrás y salto de dos en dos:

```
for ($i = 5; $i >= 1; $i--) {
    echo $i . " ";                  // → 5 4 3 2 1
}

for ($i = 0; $i <= 10; $i += 2) {
    echo $i . " ";                  // → 0 2 4 6 8 10
}
```

Ejemplo 2 — Acumular un total y contar casos dentro de un bucle:

```
$suma = 0;
$multiplosDe3 = 0;

for ($i = 1; $i <= 20; $i++) {
    $suma += $i;                    // acumulador: va sumando
    if ($i % 3 === 0) {
        $multiplosDe3++;            // contador: cuenta los que cumplen
    }
}
echo "Suma: $suma";                 // → Suma: 210
echo "Múltiplos de 3: $multiplosDe3";   // → 6
```

Consejo: Inicializa siempre el acumulador (`$suma = 0;`) o el contador **antes** del bucle. Si lo pones dentro, se reinicia en cada vuelta.

while y do...while

Repiten **mientras** una condición sea verdadera. Se usan cuando no sabes cuántas vueltas harán. `do...while` ejecuta el bloque al menos una vez.

```
while (condición) {
    // ...
}

do {
    // se ejecuta al menos una vez
} while (condición);
```

Ejemplo 1 — Repetir hasta alcanzar un valor:

```
$dinero = 100;
$anios = 0;

while ($dinero < 200) {
    $dinero = $dinero * 1.10;       // 10 % de interés anual
    $anios++;
}
echo "Se duplica en $anios años";  // → Se duplica en 8 años
```

Ejemplo 2 — Descomponer un número con % y intdiv():

```
$numero = 4829;
$cifras = 0;
$sumaCifras = 0;

while ($numero > 0) {
    $ultima = $numero % 10;        // última cifra: 9, luego 2, 8, 4
    $sumaCifras += $ultima;
    $cifras++;
    $numero = intdiv($numero, 10); // quita la última cifra
}
echo "$cifras cifras, suman $sumaCifras";   // → 4 cifras, suman 23
```

Atención: Si la condición de un `while` nunca llega a ser falsa, el bucle no termina y la página se queda colgada. Comprueba siempre que algo dentro del bucle hace avanzar la condición.

foreach

Recorre todos los elementos de un array sin necesidad de contador. Con `clave => valor` obtienes también la clave.

```
foreach ($array as $valor) { ... }
foreach ($array as $clave => $valor) { ... }
```

Ejemplo 1 — Array indexado y asociativo:

```
$frutas = ["pera", "manzana", "kiwi"];
foreach ($frutas as $fruta) {
    echo $fruta . " ";             // → pera manzana kiwi
}

$edades = ["Ana" => 21, "Luis" => 34];
foreach ($edades as $nombre => $edad) {
    echo "$nombre tiene $edad años. ";
}
```

Ejemplo 2 — Array de arrays (lista de registros):

```
$jugadores = [
    ["nombre" => "Marta", "puntos" => 18],
    ["nombre" => "Iván",  "puntos" => 25],
    ["nombre" => "Sofía", "puntos" => 11],
];

foreach ($jugadores as $j) {
    // en cada vuelta $j es UN array con "nombre" y "puntos"
    echo $j["nombre"] . ": " . $j["puntos"] . " puntos<br>";
}
```

break y continue

`break` **sale** del bucle por completo. `continue` salta a la **siguiente vuelta** sin ejecutar el resto del bloque.

Ejemplo 1 — continue para saltarse elementos:

```
for ($i = 1; $i <= 10; $i++) {
    if ($i % 2 === 0) {
        continue;                  // si es par, pasa al siguiente número
    }
    echo $i . " ";                 // → 1 3 5 7 9
}
```

Ejemplo 2 — Buscar un elemento y parar al encontrarlo:

```
$nombres = ["Ana", "Luis", "Eva", "Pablo"];
$buscado = "Eva";
$encontrado = false;

foreach ($nombres as $n) {
    if ($n === $buscado) {
        $encontrado = true;
        break;                     // ya no hace falta seguir recorriendo
    }
}
echo $encontrado ? "Está en la lista" : "No está";
```

Bucles anidados

Un bucle dentro de otro. El bucle interno completa **todas** sus vueltas por cada vuelta del externo. Sirven para filas y columnas, o para comparar cada elemento con los demás.

Ejemplo 1 — Rectángulo de asteriscos:

```
for ($fila = 1; $fila <= 3; $fila++) {
    for ($col = 1; $col <= 5; $col++) {
        echo "*";
    }
    echo "<br>";                   // salto de línea al terminar cada fila
}
```

Ejemplo 2 — Recorrer filas y columnas de una matriz:

```
$matriz = [
    [1, 2, 3],
    [4, 5, 6],
];

for ($f = 0; $f < count($matriz); $f++) {
    for ($c = 0; $c < count($matriz[$f]); $c++) {
        echo $matriz[$f][$c] . " ";   // $matriz[fila][columna]
    }
    echo "<br>";
}
```

3. Arrays

Tipos de array

Un array guarda varios valores en una sola variable. Hay tres formas habituales de usarlo:

Tipo	Ejemplo	Cómo se accede
Indexado	`["rojo", "verde"]`	Por posición, empezando en 0: `$a[0]`
Asociativo	`["nombre" => "Ana"]`	Por clave de texto: `$a["nombre"]`
Multidimensional	`[["id" => 1], ["id" => 2]]`	Varios niveles: `$a[1]["id"]`

Ejemplo 1 — Indexado y asociativo:

```
$colores = ["rojo", "verde", "azul"];
echo $colores[0];               // → rojo   (la primera posición es la 0)
echo count($colores);           // → 3

$persona = ["nombre" => "Ana", "edad" => 21];
echo $persona["nombre"];        // → Ana
$persona["edad"] = 22;          // modificar un valor
$persona["ciudad"] = "Madrid";  // añadir una clave nueva
```

Ejemplo 2 — Array de arrays (lista de registros):

```
$series = [
    ["titulo" => "Dark",  "temporadas" => 3],
    ["titulo" => "Lost",  "temporadas" => 6],
    ["titulo" => "Ozark", "temporadas" => 4],
];

echo $series[1]["titulo"];       // → Lost  (posición 1, clave "titulo")
$series[2]["temporadas"] = 5;    // modifica un dato concreto
echo count($series);             // → 3     (número de series, no de datos)
```

Añadir, modificar y eliminar

Instrucción	Qué hace
`$a[] = $x;`	Añade `$x` al final
`array_push($a, $x, $y);`	Añade uno o varios elementos al final
`array_pop($a);`	Quita y devuelve el último elemento
`array_shift($a);`	Quita y devuelve el primer elemento
`array_unshift($a, $x);`	Añade al principio
`unset($a[2]);`	Elimina la posición o clave indicada
`array_values($a);`	Devuelve el array con las posiciones renumeradas

Ejemplo 1 — Añadir y quitar elementos:

```php
$lista = ["a", "b"];
$lista[] = "c";                    // → ["a", "b", "c"]
array_push($lista, "d", "e");      // → ["a", "b", "c", "d", "e"]

$ultimo  = array_pop($lista);      // "e"   (la lista pierde el último)
$primero = array_shift($lista);    // "a"   (la lista pierde el primero)
array_unshift($lista, "z");        // → ["z", "b", "c", "d"]

// Añadir un registro nuevo a un array de arrays:
$series = [];
$series[] = ["titulo" => "Fargo", "temporadas" => 5];
```

Ejemplo 2 — Eliminar con unset() y reindexar:

```php
$tareas = ["lavar", "comprar", "estudiar"];

unset($tareas[1]);                 // elimina "comprar"
// Ahora es [0 => "lavar", 2 => "estudiar"]: las posiciones NO se renumeran

$tareas = array_values($tareas);   // → [0 => "lavar", 1 => "estudiar"]
```

Atención: Tras un `unset()` el array queda con «huecos» en las posiciones. Si luego lo recorres con un `for` y `count()`, fallará. Usa `foreach` o reindexa con `array_values()`.

Comprobar si existe algo

Función	Devuelve true cuando…
`isset($a["k"])`	La variable o clave existe y no es `null`
`empty($x)`	Está vacía: `""`, `0`, `"0"`, `null`, `false` o `[]`
`array_key_exists("k", $a)`	La clave existe, aunque su valor sea `null`
`in_array($x, $a)`	El valor `$x` está dentro del array
`array_search($x, $a)`	Devuelve la clave del valor, o `false` si no está

Ejemplo 1 — isset() y ?? para claves que pueden no existir:

```php
$stock = ["pan" => 5, "leche" => 0];

var_dump(isset($stock["pan"]));     // → true
var_dump(isset($stock["vino"]));    // → false
$cantidad = $stock["vino"] ?? 0;    // → 0, sin avisos de PHP
```

Ejemplo 2 — in_array() y array_search():

```php
$vocales = ["a", "e", "i", "o", "u"];

var_dump(in_array("e", $vocales));       // → true
var_dump(in_array("x", $vocales));       // → false

$pos = array_search("i", $vocales);      // → 2
if (array_search("a", $vocales) !== false) {
    echo "Está (en la posición 0)";
}
```

Atención: `array_search()` devuelve `0` si el valor está en la primera posición, y `0` también se considera «falso». Compara siempre con `!== false`. Y `in_array("5", [5])` es verdadero; con `in_array("5", [5], true)` el tercer parámetro exige el mismo tipo.

Funciones útiles de arrays

Función	Qué hace
`count($a)`	Número de elementos
`array_sum($a)`	Suma de todos los valores
`max($a)` `min($a)`	Valor mayor y menor
`array_keys($a)` `array_values($a)`	Solo las claves / solo los valores
`array_merge($a, $b)`	Une dos arrays en uno
`array_slice($a, inicio, cuantos)`	Devuelve un trozo (con inicio negativo cuenta desde el final)
`array_reverse($a)`	Da la vuelta al orden
`array_unique($a)`	Elimina valores repetidos
`array_column($a, "clave")`	Extrae una columna de un array de arrays
`range(1, 5)`	Crea `[1, 2, 3, 4, 5]`

Ejemplo 1 — Suma, media, máximo y mínimo:

```
$temps = [18, 22, 15, 30, 25];

echo count($temps);                          // → 5
echo array_sum($temps);                      // → 110
echo array_sum($temps) / count($temps);      // → 22    (media)
echo max($temps) . " / " . min($temps);      // → 30 / 15
```

Ejemplo 2 — Los últimos elementos, el más reciente primero:

```
$historial = ["a.php", "b.php", "c.php", "d.php", "e.php"];

$ultimos = array_slice($historial, -3);      // → ["c.php", "d.php", "e.php"]
$ultimos = array_reverse($ultimos);          // → ["e.php", "d.php", "c.php"]

$primeros = array_slice($historial, 0, 2);   // → ["a.php", "b.php"]
```

Atención: Si el array está vacío, `array_sum($a) / count($a)` divide entre 0 y PHP lanza «Division by zero». Comprueba antes que `count($a) > 0`.

Ordenar arrays

Función	Ordena…
`sort($a)` `rsort($a)`	Por valor, de menor a mayor / de mayor a menor. Renumera las posiciones
`asort($a)` `arsort($a)`	Por valor, **conservando las claves**
`ksort($a)` `krsort($a)`	Por clave
`usort($a, función)`	Con tu propio criterio (por ejemplo, por un campo de un array de arrays)

Ejemplo 1 — sort() y rsort():

```
$nums = [5, 2, 9, 1];
sort($nums);                    // $nums pasa a ser [1, 2, 5, 9]
rsort($nums);                   // $nums pasa a ser [9, 5, 2, 1]

$edades = ["Ana" => 30, "Luis" => 22];
asort($edades);                 // → ["Luis" => 22, "Ana" => 30]
```

Ejemplo 2 — usort() con el operador <=>:

```
$personas = [
    ["nombre" => "Ana",  "edad" => 30],
    ["nombre" => "Luis", "edad" => 22],
    ["nombre" => "Eva",  "edad" => 27],
];

// <=> devuelve -1, 0 o 1 según $a sea menor, igual o mayor que $b
usort($personas, fn($a, $b) => $a["edad"] <=> $b["edad"]);   // de menor a mayor
usort($personas, fn($a, $b) => $b["edad"] <=> $a["edad"]);   // de mayor a menor
```

Atención: Estas funciones **modifican el array original** y devuelven `true`, no el array ordenado. `$orden = sort($a);` guarda `true` en `$orden`. Si necesitas conservar el original, ordena una copia: `$copia = $a; sort($copia);`. Y si un ejercicio prohíbe usar estas funciones, tendrás que ordenar con bucles.

array_map() y array_filter()

Reciben una función. **`array_map`** transforma cada elemento y devuelve un array del mismo tamaño. **`array_filter`** se queda solo con los elementos para los que la función devuelve `true`, y **conserva las claves originales** (usa `array_values` si necesitas renumerarlas). Las funciones flecha `fn(...) =>` se explican en la sección de funciones.

Ejemplo 1 — Transformar y filtrar números:

```
$precios = [10, 25, 8];

$conIva = array_map(fn($p) => $p * 1.21, $precios);   // → [12.1, 30.25, 9.68]
$caros  = array_filter($precios, fn($p) => $p > 9);   // → [0 => 10, 1 => 25]
```

Ejemplo 2 — Con arrays de arrays:

```
$personas = [
    ["nombre" => "Ana",  "edad" => 30],
    ["nombre" => "Luis", "edad" => 22],
    ["nombre" => "Eva",  "edad" => 27],
];

$nombres = array_column($personas, "nombre");              // → ["Ana", "Luis",
"Eva"]

$mayores = array_filter($personas, fn($p) => $p["edad"] >= 25);
// conserva las claves: [0 => Ana, 2 => Eva]
$mayores = array_values($mayores);                         // → [0 => Ana, 1 =>
Eva]
```

Técnicas habituales con arrays

Casi todos los ejercicios con arrays se resuelven combinando unas pocas técnicas. Aprende el patrón, no el ejemplo: lo único que cambia de un problema a otro son los nombres de las claves y la condición. En todos los ejemplos usamos este array:

```
$jugadores = [
    ["nombre" => "Marta", "equipo" => "Rojo", "puntos" => 18],
    ["nombre" => "Iván",  "equipo" => "Azul", "puntos" => 25],
    ["nombre" => "Sofía", "equipo" => "Rojo", "puntos" => 11],
    ["nombre" => "Pablo", "equipo" => "Azul", "puntos" => 25],
];
```

1. Acumular un valor de cada elemento

Ejemplo 1 — Sumar todos los puntos:

```
$total = 0;                          // el acumulador se inicializa ANTES del
bucle
foreach ($jugadores as $j) {
    $total += $j["puntos"];
}
echo $total;                         // → 79
```

Ejemplo 2 — Sumar solo los que cumplen una condición:

```
$totalRojo = 0;
foreach ($jugadores as $j) {
    if ($j["equipo"] === "Rojo") {
        $totalRojo += $j["puntos"];
    }
}
echo $totalRojo;                     // → 29
```

2. Contar y filtrar

Ejemplo 1 — Contar los que cumplen una condición:

```
$destacados = 0;
foreach ($jugadores as $j) {
    if ($j["puntos"] >= 20) {
        $destacados++;
    }
}
echo $destacados;                    // → 2
```

Ejemplo 2 — Construir un array nuevo solo con los que cumplen:

```
$nombresDestacados = [];
foreach ($jugadores as $j) {
    if ($j["puntos"] >= 20) {
        $nombresDestacados[] = $j["nombre"];     // se añade al array nuevo
    }
}
echo implode(", ", $nombresDestacados);          // → Iván, Pablo
echo count($nombresDestacados);                  // → 2
```

3. Máximo y mínimo guardando el elemento completo

Ejemplo 1 — El jugador con más puntos:

```
$mejor = null;
foreach ($jugadores as $j) {
    if ($mejor === null || $j["puntos"] > $mejor["puntos"]) {
        $mejor = $j;                // se guarda el jugador entero, no solo los
puntos
    }
}
echo $mejor["nombre"] . " (" . $mejor["puntos"] . ")";   // → Iván (25)
```

Ejemplo 2 — Máximo y mínimo a la vez, y qué pasa en un empate:

```
$mejor = $jugadores[0];             // se parte del primer elemento
$peor  = $jugadores[0];

foreach ($jugadores as $j) {
    if ($j["puntos"] > $mejor["puntos"]) { $mejor = $j; }   // con > se queda el
primero en empate
    if ($j["puntos"] < $peor["puntos"])  { $peor  = $j; }
}
echo $mejor["nombre"];              // → Iván  (Pablo también tiene 25, pero
Iván va antes)
echo $peor["nombre"];               // → Sofía
```

Consejo: Para que en caso de empate gane el **primero** de la lista usa `>` o `<`. Con `>=` o `<=` ganaría el último. Si parte de `$jugadores[0]`, comprueba antes que el array no está vacío.

4. Contar y agrupar por clave

Ejemplo 1 — Cuántos jugadores hay en cada equipo:

```
$porEquipo = [];
foreach ($jugadores as $j) {
    $equipo = $j["equipo"];
    $porEquipo[$equipo] = ($porEquipo[$equipo] ?? 0) + 1;   // si no existe,
parte de 0
}
// → ["Rojo" => 2, "Azul" => 2]
```

Ejemplo 2 — Agrupar los nombres de cada equipo:

```
$grupos = [];
foreach ($jugadores as $j) {
    $grupos[$j["equipo"]][] = $j["nombre"];    // crea el array del equipo si aún
no existe
}
// → ["Rojo" => ["Marta", "Sofía"], "Azul" => ["Iván", "Pablo"]]

foreach ($grupos as $equipo => $nombres) {
    echo "$equipo: " . implode(", ", $nombres) . " (" . count($nombres) .
")<br>";
}
```

5. Buscar un elemento por su id

Ejemplo 1 — Recorrer y parar al encontrarlo:

```
$libros = [
    ["id" => 1, "titulo" => "Dune"],
    ["id" => 2, "titulo" => "Emma"],
    ["id" => 3, "titulo" => "Ulises"],
];

$id = 2;
$encontrado = null;                  // null significa «todavía no encontrado»
foreach ($libros as $libro) {
    if ($libro["id"] === $id) {
        $encontrado = $libro;
        break;
    }
}
```

Ejemplo 2 — Cuando el id llega por la URL:

```
// Un id que viene de $_GET es TEXTO: "2". Hay que convertirlo antes de comparar.
$id = (int) ($_GET["id"] ?? 0);      // si no llega o no es válido, queda en 0

$encontrado = null;
foreach ($libros as $libro) {
    if ($libro["id"] === $id) { $encontrado = $libro; break; }
}

if ($encontrado === null) {
    echo "Libro no encontrado";
} else {
    echo $encontrado["titulo"];
}
```

6. Proteger el código frente a arrays vacíos y resultados nulos

Ejemplo 1 — Comprobar antes de calcular:

```
$lista = [];

if (count($lista) === 0) {
    echo "No hay datos";
} else {
    echo array_sum($lista) / count($lista);
}
```

Ejemplo 2 — Comprobar que hay resultado antes de usarlo:

```
$mejor = null;                                  // p. ej. lo que devuelve una
búsqueda sin éxito

echo $mejor["nombre"];                          // MAL: Warning: Trying to
access array offset on value of type null

if ($mejor !== null) {                          // BIEN: se comprueba primero
    echo $mejor["nombre"];
} else {
    echo "Sin resultados";
}
```

4. Funciones

Crear y llamar funciones

Una función agrupa código que puedes reutilizar. Recibe **parámetros**, y con `return` devuelve un resultado y termina su ejecución. Los tipos de parámetros y de retorno (`int`, `float`, `string`, `bool`, `array`, `void`) son opcionales pero ayudan a detectar errores. Un `?` delante del tipo (`?array`) permite también `null`.

```
function nombreFuncion($parametro1, $parametro2) {
    // código
    return $resultado;
}
```

Ejemplo 1 — Función con return:

```
function saludar($nombre) {
    return "Hola, " . $nombre;
}
echo saludar("Ana");              // → Hola, Ana

function esPar($n) {
    return $n % 2 === 0;          // devuelve true o false
}
var_dump(esPar(8));               // → bool(true)
```

Ejemplo 2 — Valores por defecto y tipos:

```
function precioFinal(float $precio, int $descuento = 0): float {
    return $precio - ($precio * $descuento / 100);
}

echo precioFinal(50);             // → 50   (descuento por defecto: 0)
echo precioFinal(50, 20);         // → 40
```

Atención: Lo que escribas después de un `return` no se ejecuta. Una función que no tiene `return` devuelve `null`.

Devolver varios valores

Una función solo puede devolver un valor, pero ese valor puede ser un **array** con todo lo que necesites.

Ejemplo 1 — Devolver un array asociativo:

```
function resumen(array $numeros): array {
    return [
        "suma"  => array_sum($numeros),
        "media" => round(array_sum($numeros) / count($numeros), 2),
        "max"   => max($numeros),
    ];
}

$r = resumen([4, 8, 6]);
echo $r["media"];                 // → 6
echo $r["max"];                   // → 8
```

Ejemplo 2 — Desestructurar el resultado y devolver null si no hay resultado:

```
function rango(array $numeros): ?array {
    if (count($numeros) === 0) {
        return null;              // sin datos: se avisa con null
    }
    return ["min" => min($numeros), "max" => max($numeros)];
}

$r = rango([4, 9, 2]);
if ($r !== null) {
    ["min" => $minimo, "max" => $maximo] = $r;   // saca cada valor a su variable
    echo "$minimo - $maximo";                    // → 2 - 9
}
```

Ámbito (scope) de las variables

Dentro de una función solo existen sus parámetros y las variables que crea ella misma. Las variables de fuera **no se ven** (las constantes sí), y las de dentro **desaparecen** cuando la función termina.

Ejemplo 1 — Una variable de fuera no entra en la función:

```
$iva = 21;

function conIva($precio) {
    return $precio * (1 + $iva / 100);       // MAL: $iva no existe aquí:
Undefined variable
}

function conIvaBien($precio, $iva) {
    return $precio * (1 + $iva / 100);       // BIEN: se pasa como parámetro
}
echo conIvaBien(100, $iva);                  // → 121
```

Ejemplo 2 — Una variable de dentro no sale de la función:

```
function calcular() {
    $resultado = 10;                         // variable local
}
calcular();
echo $resultado;                             // MAL: Undefined variable

function calcularBien() {
    return 10;
}
$resultado = calcularBien();                 // BIEN: se devuelve con return y se
guarda fuera
echo $resultado;                             // → 10
```

Consejo: Existe la palabra `global` para «entrar» a una variable de fuera, pero no la uses: pasa los datos como parámetros y devuelve los resultados con `return`.

Paso por valor y por referencia

Por defecto los valores se **copian** al pasarlos a una función: lo que hagas dentro no cambia el original. Si escribes `&` delante del parámetro, la función trabaja con el original.

Ejemplo 1 — Por valor: el original no cambia:

```
function agregarCero(array $lista) {
    $lista[] = 0;                // modifica SU copia
    return $lista;
}

$a = [1, 2];
$b = agregarCero($a);
// $a sigue siendo [1, 2]
// $b es [1, 2, 0]
```

Ejemplo 2 — Ordenar una copia y por referencia con &:

```
function ordenada(array $lista): array {
    sort($lista);                // sort() ordena la copia local
    return $lista;               // el original se queda como estaba
}
$original = [3, 1, 2];
$orden = ordenada($original);    // $original → [3, 1, 2]   $orden → [1, 2, 3]

function vaciar(array &$lista) {  // & = se trabaja con el original
    $lista = [];
}
vaciar($original);               // $original → []
```

Funciones anónimas y funciones flecha

Una función sin nombre se puede guardar en una variable o pasar a otra función (como en `array_map` o `usort`). La **función flecha** `fn(parámetros) => expresión` es la versión corta y lee automáticamente las variables de fuera.

Ejemplo 1 — Guardar una función en una variable:

```
$doble = fn($n) => $n * 2;
echo $doble(4);                  // → 8

$saludo = function ($nombre) {
    return "Hola $nombre";
};
echo $saludo("Eva");             // → Hola Eva
```

Ejemplo 2 — Usar variables de fuera dentro de la función:

```
$minimo = 5;

// La función flecha ve $minimo automáticamente:
$a = array_filter([3, 7, 9], fn($n) => $n >= $minimo);

// La función normal necesita "use" para ver variables de fuera:
$b = array_filter([3, 7, 9], function ($n) use ($minimo) {
    return $n >= $minimo;
});
```

Funciones de texto

Función	Qué hace	Ejemplo → resultado
`strlen($t)`	Longitud en bytes	`strlen("Hola")` → 4
`mb_strlen($t)`	Longitud en caracteres (con tildes y ñ)	`mb_strlen("Año")` → 3
`strtolower` `strtoupper`	Pasa a minúsculas / mayúsculas	`strtolower("HOLA")` → hola
`mb_strtolower` `mb_strtoupper`	Igual, pero con tildes y ñ	`mb_strtolower("ÁRBOL")` → árbol
`ucfirst` `ucwords`	Primera letra / primera de cada palabra en mayúscula	`ucwords("ana lópez")` → Ana López
`trim` `ltrim` `rtrim`	Quita espacios del principio y del final	`trim("  hola ")` → hola
`str_contains($t, $x)`	Comprueba si el texto contiene `$x`	`str_contains("Hola mundo", "mundo")` → true
`str_starts_with` `str_ends_with`	Empieza / termina por	`str_starts_with("Hola", "Ho")` → true
`strpos` `stripos`	Posición de un texto (`false` si no está). `stripos` no distingue mayúsculas	`strpos("Hola", "la")` → 2
`str_replace($a, $b, $t)`	Reemplaza `$a` por `$b`	`str_replace("a", "o", "gato")` → goto
`substr($t, inicio, largo)`	Extrae un trozo	`substr("Hola mundo", 0, 4)` → Hola
`explode($sep, $t)`	Divide un texto en un array	`explode(",", "a,b,c")` → ["a","b","c"]
`implode($sep, $array)`	Une un array en un texto	`implode(" - ", ["a","b"])` → a - b
`str_pad($t, n, "0", STR_PAD_LEFT)`	Rellena hasta una longitud	`str_pad("7", 2, "0", STR_PAD_LEFT)` → 07
`str_word_count($t)`	Número de palabras	`str_word_count("uno dos tres")` → 3
`strrev($t)`	Da la vuelta al texto	`strrev("abc")` → cba
`date("d/m/Y")`	Fecha actual con el formato indicado	→ 02/11/2026

Ejemplo 1 — Normalizar un texto:

```
$entrada = "   mARÍA lópez  ";

$limpio = trim($entrada);                    // "mARÍA lópez"
$nombre = ucwords(mb_strtolower($limpio));     // "María López"

echo $nombre . " (" . mb_strlen($nombre) . " caracteres)";   // → María López (11 caracteres)
```

Ejemplo 2 — Buscar, dividir y reordenar fechas:

```php
// Buscar sin distinguir mayúsculas ni tildes mal tratadas:
$titulo = "El Señor de los Anillos";
var_dump(str_contains(mb_strtolower($titulo), mb_strtolower("SEÑOR")));   // → true

// Dividir una fecha y guardar cada parte en su variable:
$f = "1998-03-07";
[$anio, $mes, $dia] = explode("-", $f);        // "1998", "03", "07"
echo "$dia/$mes/$anio";                        // → 07/03/1998

// Otra forma, con date() y strtotime():
echo date("d/m/Y", strtotime($f));             // → 07/03/1998
```

Atención: `strpos()` devuelve `0` si el texto está al principio, y `0` cuenta como «falso». Compara siempre con `!== false`, o usa `str_contains()`, que devuelve `true` o `false` directamente.

Atención: Con textos con tildes o ñ usa las versiones `mb_`. `strlen("Año")` da 4 (cuenta bytes) y `strtolower("Á")` no cambia la letra.

Funciones numéricas y de formato

Función	Qué hace	Ejemplo → resultado
`round($n, dec)`	Redondea al más cercano	`round(7.456, 2)` → 7.46
`floor($n)`	Redondea hacia abajo	`floor(4.9)` → 4
`ceil($n)`	Redondea hacia arriba	`ceil(4.1)` → 5
`abs($n)`	Valor absoluto	`abs(-7)` → 7
`intdiv($a, $b)`	División entera	`intdiv(17, 5)` → 3
`$a % $b`	Resto de la división	`17 % 5` → 2
`max(...)` `min(...)`	El mayor / menor de varios valores	`max(3, 9, 4)` → 9
`rand($a, $b)`	Entero aleatorio entre `$a` y `$b`	`rand(1, 6)` → 1 al 6
`is_numeric($x)`	Comprueba si es un número o un texto numérico	`is_numeric("12.5")` → true
`intval($x)` `floatval($x)`	Convierte a entero / decimal	`floatval("3.5")` → 3.5
`number_format($n, dec, sepDec, sepMiles)`	Da formato a un número (devuelve texto)	Ver el ejemplo siguiente
`sprintf("%.2f", $n)`	Texto con formato; `%02d` rellena con ceros	`sprintf("%02d", 7)` → 07

Ejemplo 1 — Redondeos:

```php
echo round(2.5);           // → 3
echo round(7.456, 2);      // → 7.46
echo floor(4.9);           // → 4
echo ceil(4.1);            // → 5

echo ceil(150 / 60);       // → 3   (150 minutos son 3 horas «iniciadas»)
```

Ejemplo 2 — Dar formato a importes y a duraciones:

```
$importe = 1234.5;
echo number_format($importe, 2);              // → 1,234.50   (miles con coma,
decimales con punto)
echo number_format($importe, 2, ",", ".");    // → 1.234,50   (formato español)
echo number_format($importe, 2, ".", "");     // → 1234.50    (sin separador de
miles)

// Pasar segundos a horas:minutos:segundos
$seg = 3725;
$h = intdiv($seg, 3600);                      // → 1
$m = intdiv($seg % 3600, 60);                 // → 2
$s = $seg % 60;                               // → 5
echo sprintf("%02d:%02d:%02d", $h, $m, $s);   // → 01:02:05
```

Consejo: `number_format()` redondea y devuelve **texto**. Sigue calculando con el número original y da formato solo en el momento de mostrarlo.

5. PHP y HTML

<?= ?> y la sintaxis alternativa

`<?= $x ?>` es la forma corta de `<?php echo $x; ?>`: **muestra** un valor. Para la lógica (condiciones y bucles) se usa `<?php ... ?>`. Dentro del HTML resulta más legible la **sintaxis alternativa**, que sustituye las llaves por `:` y un cierre con nombre propio.

Con llaves	Sintaxis alternativa
`if (...) { }`	`if (...): ... endif;`
`} else {`	`else:`
`} elseif (...) {`	`elseif (...):`
`foreach (...) { }`	`foreach (...): ... endforeach;`
`for (...) { }`	`for (...): ... endfor;`
`while (...) { }`	`while (...): ... endwhile;`

Ejemplo 1 — Una lista con foreach:

```
<ul>
<?php foreach ($frutas as $fruta): ?>
    <li><?= $fruta ?></li>
<?php endforeach; ?>
</ul>
```

Ejemplo 2 — if / else, mensaje de lista vacía y clase según el estado:

```
<?php if (count($pelis) === 0): ?>
    <p>No hay películas.</p>
<?php else: ?>
    <table>
        <?php foreach ($pelis as $p): ?>
            <tr class="<?= $p["vista"] ? "vista" : "pendiente" ?>">
                <td><?= $p["titulo"] ?></td>
                <td><?= $p["vista"] ? "Vista" : "Pendiente" ?></td>
            </tr>
        <?php endforeach; ?>
    </table>
<?php endif; ?>
```

Atención: Dentro de `<?= ?>` solo cabe **una expresión** (un valor, un cálculo o un ternario). No se puede escribir un `if` ni un `foreach`. Y cada `if:` o `foreach:` abierto debe cerrarse con su `endif;` / `endforeach;`; si falta alguno, PHP da «Parse error: unexpected end of file».

Lógica arriba, HTML abajo

Una buena costumbre: calcula todo al **principio** del archivo y deja el HTML solo para mostrar. Así el HTML queda limpio y los errores son más fáciles de encontrar.

Ejemplo 1 — Calcular antes de pintar:

```
<?php
// 1) LÓGICA
$total = 0;
foreach ($items as $i) {
    $total += $i["precio"];
}
$envio = $total > 100 ? "Envío gratis" : "Envío 5 €";
?>
<!-- 2) HTML: solo muestra -->
<p>Total: <?= number_format($total, 2) ?> €</p>
<p><?= $envio ?></p>
```

Ejemplo 2 — Mostrar booleanos y valores opcionales:

```
<p>Vegetariana: <?= $receta["vegetariana"] ? "Sí" : "No" ?></p>
<p>Notas: <?= $receta["notas"] ?? "Sin notas" ?></p>
<p>Tiempo: <?= $receta["minutos"] ?> min (<?= round($receta["minutos"] / 60, 1) ?
> h)</p>
```

Atención: `<?= true ?>` imprime `1` y `<?= false ?>` no imprime nada. Para mostrar «Sí» o «No», usa siempre un ternario.

Atributos dinámicos: class, selected, checked y value

Puedes escribir PHP dentro de los atributos de una etiqueta para cambiar su aspecto o su estado según los datos.

Ejemplo 1 — Clase CSS según una condición:

```
<span class="etiqueta <?= $stock > 0 ? "verde" : "roja" ?>">
    <?= $stock > 0 ? "Disponible" : "Agotado" ?>
</span>
```

Ejemplo 2 — Un formulario que recuerda lo que se eligió:

```
<?php
$categorias = ["entrante", "principal", "postre"];
$elegida = $_GET["cat"] ?? "";
?>
<form method="GET">
    <select name="cat">
        <option value="">Todas</option>
        <?php foreach ($categorias as $c): ?>
            <option value="<?= $c ?>" <?= $elegida === $c ? "selected" : "" ?>><?
= $c ?></option>
        <?php endforeach; ?>
    </select>

    <input type="text" name="q" value="<?= htmlspecialchars($_GET["q"] ?? "") ?
>">
    <input type="checkbox" name="veg" <?= isset($_GET["veg"]) ? "checked" : "" ?
>> Vegetariana
    <button type="submit">Buscar</button>
</form>
```

Seguridad: htmlspecialchars()

Todo lo que escribe un usuario puede contener HTML o JavaScript. Si lo imprimes tal cual, ese código se ejecuta en el navegador. `htmlspecialchars()` convierte los caracteres especiales (`<`, `>`, `"`, `&`) en texto inofensivo.

Ejemplo 1 — Qué hace la función:

```
$texto = "<b>hola</b>";
echo htmlspecialchars($texto);     // se ve en pantalla como: <b>hola</b>  (como
texto, sin negrita)
```

Ejemplo 2 — Imprimir lo que viene de un formulario:

```
$nombre = $_POST["nombre"] ?? "";

echo "Hola, " . $nombre;                        // MAL: peligroso
echo "Hola, " . htmlspecialchars($nombre);      // BIEN: seguro

// en HTML:
// <p>Hola, <?= htmlspecialchars($nombre) ?></p>
```

Consejo: Aplica `htmlspecialchars()` a **todo** valor que venga del usuario antes de mostrarlo (`$_GET`, `$_POST`, datos de sesión escritos por él).

6. Organizar el proyecto: include y require

include y require

Cargan otro archivo PHP. El archivo incluido se ejecuta **como si su contenido estuviera escrito en ese punto**, así que comparte las variables del archivo que lo carga (no hace falta importar nada).

Instrucción	Si el archivo no existe...	Úsala para...
`require "x.php";`	Error fatal: el script se detiene	Archivos imprescindibles: datos, configuración, funciones, cabecera
`include "x.php";`	Aviso, y el script continúa	Partes opcionales: un componente, un widget
`require_once "x.php";`	Igual que `require`, pero no lo carga dos veces	Funciones y clases (evita «Cannot redeclare»)

Ejemplo 1 — Cargar configuración, datos y funciones:

```
// index.php
require "config.php";       // constantes
require "datos.php";        // arrays con los datos ($productos, $usuarios...)
require "funciones.php";    // funciones propias

// A partir de aquí ya están disponibles las constantes, los arrays y las
funciones
echo count($productos);
```

Ejemplo 2 — Rutas desde una subcarpeta:

```
// controllers/guardar.php  (el archivo está DENTRO de controllers/)
require "../config.php";            // ../ = subir una carpeta
require "../datos.php";

header("Location: ../index.php");   // la redirección también necesita ../
exit();
```

Atención: Si recibes «Failed opening required», casi siempre es la ruta. Piensa desde qué carpeta se está ejecutando el archivo: dentro de `controllers/` hay que subir con `../`.

Partials: cabecera y pie compartidos

Un **partial** es un trozo de HTML (con o sin PHP) que se repite en varias páginas: cabecera, menú, pie. Se escribe una vez y se incluye con `require`. Si cambias el menú, cambia en todas las páginas.

Ejemplo 1 — Una página que usa header y footer:

```
<?php
require "datos.php";
// lógica de la página...
?>
<?php require "partials/header.php"; ?>

<main>
    <h1>Mi página</h1>
</main>

<?php require "partials/footer.php"; ?>
```

Ejemplo 2 — Qué contienen header.php y footer.php:

```
<!-- partials/header.php: abre el documento y NO lo cierra -->
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mi sitio</title>
    <link rel="stylesheet" href="assets/css/styles.css">
</head>
<body>
<header><h1>Mi sitio</h1></header>

<!-- partials/footer.php: cierra lo que abrió el header -->
<footer>© 2026 Mi sitio</footer>
</body>
</html>
```

Atención: `session_start()` debe ejecutarse antes de enviar cualquier HTML. Si usas sesiones, no lo escribas dentro de `header.php` (que ya imprime HTML): ponlo en la primera línea de cada página, antes de incluir el header.

Componentes dentro de un bucle

Un partial también puede ser un **componente**: un trozo de HTML que se incluye una vez por cada elemento. Como el archivo incluido comparte las variables, puede usar la variable del `foreach` donde se incluye.

Ejemplo 1 — Una tarjeta por cada elemento:

```
<!-- index.php -->
<?php foreach ($recetas as $receta): ?>
    <?php include "partials/tarjeta.php"; ?>
<?php endforeach; ?>

<!-- partials/tarjeta.php  (usa $receta, que existe en cada vuelta) -->
<div class="tarjeta">
    <h3><?= $receta["titulo"] ?></h3>
    <p><?= $receta["minutos"] ?> min</p>
</div>
```

Ejemplo 2 — Lógica al principio del componente:

```
<?php
// partials/tarjeta.php
// La lógica va arriba, como en cualquier otro archivo
$etiqueta = $receta["minutos"] <= 20 ? "Rápida" : "Normal";
$clase    = $receta["minutos"] <= 20 ? "verde" : "gris";
?>
<div class="tarjeta">
    <h3><?= $receta["titulo"] ?></h3>
    <span class="etiqueta <?= $clase ?>"><?= $etiqueta ?></span>
</div>
```

Atención: El componente depende del nombre de la variable del bucle. Si en `index.php` escribes `foreach ($recetas as $r)`, el partial tendría que usar `$r`. Usa el mismo nombre en los dos sitios.

Estructura de carpetas recomendada

```
/proyecto
├── index.php
├── config.php          ← constantes (define)
├── datos.php           ← arrays con los datos
├── funciones.php       ← funciones propias
├── controllers/        ← archivos que procesan formularios (sin HTML)
├── partials/           ← trozos de HTML reutilizables
├── classes/            ← una clase por archivo (POO)
└── assets/
    ├── css/
    └── img/
```

Consejo: Si el enunciado de un ejercicio indica la estructura de archivos, respétala exactamente: nombres, carpetas y mayúsculas incluidos.

7. Formularios, GET y POST

Cómo funciona un formulario

Un formulario HTML recoge datos y los envía a un archivo PHP. PHP los recibe en dos arrays superglobales: `$_GET` y `$_POST`. Las superglobales se pueden usar en cualquier sitio, incluso dentro de funciones.

	GET	POST
Los datos viajan…	En la URL: `pagina.php?q=pan&cat=postre`	En el cuerpo de la petición
Se ven en la URL	Sí	No
Se pueden guardar en favoritos	Sí	No
Se usa para…	Búsquedas y filtros	Crear, modificar o borrar datos; login
Se leen con…	`$_GET["campo"]`	`$_POST["campo"]`

```
<form action="controllers/guardar.php" method="POST">
    <input type="text" name="titulo">
    <button type="submit">Enviar</button>
</form>
```

Atención: Sin el atributo `name`, el campo **no se envía**. El nombre que pongas en `name` es la clave con la que lo recibirás: `name="titulo"` → `$_POST["titulo"]`.

Ejemplo 1 — Un formulario que se procesa en la misma página:

```
<?php
$mensaje = "";

// La primera vez que se carga la página NO hay formulario enviado
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nombre = trim($_POST["nombre"] ?? "");
    $mensaje = "Hola, " . htmlspecialchars($nombre);
}
?>
<form method="POST">                    <!-- sin action: se envía a la misma
página -->
    <input type="text" name="nombre">
    <button type="submit">Saludar</button>
</form>
<p><?= $mensaje ?></p>
```

Ejemplo 2 — Un filtro GET: varios filtros combinados:

```
$max    = $_GET["max"] ?? "";            // minutos máximos (puede llegar vacío)
$soloVeg = isset($_GET["veg"]);          // el checkbox solo llega si está
marcado

$resultado = [];
foreach ($recetas as $r) {
    // Un filtro vacío NO filtra: se considera que «cumple» automáticamente
    $cumpleTiempo = $max === "" || $r["minutos"] <= (int) $max;
    $cumpleVeg    = !$soloVeg || $r["vegetariana"];

    if ($cumpleTiempo && $cumpleVeg) {
        $resultado[] = $r;               // solo pasan las que cumplen TODOS los
filtros activos
    }
}
```

Consejo: La idea clave del filtro combinado: para cada filtro, `filtroVacío || cumpleCondición`. Si el filtro no está activo, el elemento pasa; si está activo, tiene que cumplir. Después se unen todos con `&&`.

Tipos de campos

Campo	Qué llega a PHP
`text` `number` `email` `password` `textarea`	Un **texto** (también los `number`: `"25"`, no `25`)
`select`	El `value` de la opción elegida
`radio`	El `value` del radio marcado (si no hay ninguno, el campo no existe)
`checkbox`	Solo existe si está **marcado** (por defecto con el valor `"on"`)
`hidden`	El valor fijo que tú pongas (útil para enviar un `id`)

Ejemplo 1 — Checkbox y select:

```
// <input type="checkbox" name="disponible">
$disponible = isset($_POST["disponible"]);          // true si está marcado,
false si no

// <select name="tipo"> ... </select>
$tipo = $_POST["tipo"] ?? "";

// <input type="number" name="edad" step="0.1">
$edad = (float) ($_POST["edad"] ?? 0);               // se convierte a número
para operar
```

Ejemplo 2 — Varios checkbox con el mismo nombre:

```
// <input type="checkbox" name="extras[]" value="queso">
// <input type="checkbox" name="extras[]" value="bacon">
// <input type="checkbox" name="extras[]" value="huevo">

$extras = $_POST["extras"] ?? [];                    // llega un ARRAY con los
marcados
echo count($extras) . " extras: " . implode(", ", $extras);   // → 2 extras:
queso, huevo
```

Validar los datos

Nunca te fíes de lo que llega: valida siempre en PHP, aunque el HTML ya tenga `required` o `min`.
Comprobaciones habituales:

Qué compruebo	Cómo
Campo vacío	`trim($x) === ""`
Es un número	`is_numeric($x)`
Está en un rango	`$n >= 1 && $n <= 10`
Longitud mínima	`mb_strlen(trim($x)) >= 3`
Valor permitido	`in_array($x, ["interior", "terraza"], true)`
Es un email	`filter_var($x, FILTER_VALIDATE_EMAIL) !== false`

Ejemplo 1 — Comprobaciones en orden: gana el primer error:

```
$nombre = trim($_POST["nombre"] ?? "");
$edad   = trim($_POST["edad"] ?? "");

$error = "";
if ($nombre === "" || $edad === "") {
    $error = "campos";                      // 1.º: ¿falta algo?
} elseif (mb_strlen($nombre) < 3) {
    $error = "nombre";                      // 2.º: ¿nombre demasiado corto?
} elseif (!is_numeric($edad)) {
    $error = "edad_formato";                // 3.º: ¿es un número?
} elseif ($edad < 0 || $edad > 120) {
    $error = "edad_rango";                  // 4.º: ¿está en el rango?
}

if ($error !== "") {
    header("Location: index.php?error=$error");
    exit();
}
```

Ejemplo 2 — Acumular todos los errores en una lista:

```
$errores = [];

if (mb_strlen($nombre) < 3) {
    $errores[] = "El nombre debe tener al menos 3 caracteres";
}
if (!is_numeric($edad) || $edad < 0 || $edad > 120) {
    $errores[] = "La edad no es válida";
}

if (count($errores) === 0) {
    echo "Todo correcto";
} else {
    foreach ($errores as $e) {
        echo "<p class='error'>$e</p>";
    }
}
```

Atención: `empty()` considera vacíos el `0` y `"0"`. Si el valor 0 puede ser válido (o quieres darle un mensaje distinto, como «debe ser mayor que 0»), no uses `empty()`: comprueba `trim($x) === ""` primero y `is_numeric()` y el rango después. Ten en cuenta también que `strlen()` cuenta bytes: con tildes o ñ usa `mb_strlen()`.

Redirecciones: header() y exit

`header("Location: archivo.php")` ordena al navegador ir a otra página. Debe ejecutarse **antes de enviar cualquier HTML** y siempre va seguido de `exit()`, que detiene el script. Sin `exit()`, PHP seguiría ejecutando el código de debajo aunque el navegador ya se esté yendo.

Ejemplo 1 — Redirigir:

```
header("Location: dashboard.php");
exit();
```

Ejemplo 2 — Enviar un código de error por la URL y mostrar su mensaje:

```
// controllers/guardar.php
header("Location: ../index.php?error=campos");
exit();

// index.php: un array relaciona cada código con su mensaje
$mensajes = [
    "campos" => "Rellena todos los campos",
    "edad"   => "La edad no es válida",
];
$error = $_GET["error"] ?? "";

// en el HTML:
// <?php if (isset($mensajes[$error])): ?>
//     <p class="error"><?= $mensajes[$error] ?></p>
// <?php endif; ?>
```

Atención: «Cannot modify header information - headers already sent» significa que se envió algo al navegador antes del `header()`: HTML, un `echo` o incluso un espacio o línea en blanco antes de `<?php`. Pon la lógica y las redirecciones al principio del archivo.

El patrón formulario → controller → redirección

Un **controller** es un archivo PHP sin HTML cuyo único trabajo es recibir un formulario, validar, procesar y redirigir. La página con el formulario solo se encarga de mostrar.

```
1. index.php            muestra el formulario
2. controllers/x.php    recibe el POST, valida y procesa
3. header() + exit()    redirige otra vez a index.php
4. index.php            muestra el resultado o el mensaje de error
```

Ejemplo 1 — Estructura de un controller:

```php
<?php
session_start();
require "../config.php";

// 1. Solo se acepta POST: si entran por la URL, se les devuelve al inicio
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    header("Location: ../index.php");
    exit();
}

// 2. Recoger los datos
$importe = trim($_POST["importe"] ?? "");

// 3. Validar: si algo falla, se avisa y se sale
if ($importe === "" || !is_numeric($importe) || $importe <= 0) {
    header("Location: ../index.php?error=importe");
    exit();
}

// 4. Procesar y guardar (por ejemplo, en la sesión)
// ...

// 5. Redirigir siempre al final
header("Location: ../index.php");
exit();
```

Ejemplo 2 — Borrar con un botón: POST con un campo oculto:

```php
<!-- En index.php, una fila por elemento -->
<form action="controllers/eliminar.php" method="POST">
    <input type="hidden" name="id" value="<?= $item["id"] ?>">
    <button type="submit">Eliminar</button>
</form>

// controllers/eliminar.php
$id = (int) ($_POST["id"] ?? 0);        // llega como texto: se convierte a
número
```

Consejo: Las acciones que modifican datos (crear, borrar, vaciar) deben ir por POST. Un enlace GET puede ejecutarse sin querer, por ejemplo al recargar la página.

8. Sesiones

Qué es una sesión

PHP «olvida» todo al terminar cada petición: las variables desaparecen. Una **sesión** permite guardar datos en el servidor y recuperarlos en las siguientes páginas del mismo usuario. Se guardan en el array superglobal `$_SESSION`.

Instrucción	Para qué sirve
`session_start();`	Abre la sesión. Va **en la primera línea** de cada archivo que use `$_SESSION`
`$_SESSION["clave"] = valor;`	Guarda un dato
`$_SESSION["clave"] ?? valor`	Lee un dato (con valor por defecto si no existe)
`isset($_SESSION["clave"])`	Comprueba si existe
`unset($_SESSION["clave"]);`	Borra un dato concreto
`session_destroy();`	Destruye toda la sesión

Ejemplo 1 — Un contador de visitas:

```
<?php
session_start();

$_SESSION["visitas"] = ($_SESSION["visitas"] ?? 0) + 1;
?>
<p>Has visitado esta página <?= $_SESSION["visitas"] ?> veces.</p>
```

Ejemplo 2 — Inicializar la sesión solo la primera vez:

```
<?php
session_start();

// Si todavía no existe, se crea con su valor inicial
if (!isset($_SESSION["carrito"])) {
    $_SESSION["carrito"] = [];
}

// A partir de aquí, $_SESSION["carrito"] siempre existe
$cantidad = count($_SESSION["carrito"]);
```

Atención: Si olvidas `session_start()`, `$_SESSION` estará vacío y no verás ningún error: simplemente «no se guarda nada». Si ves «headers already sent», recuerda que `session_start()` también debe ir antes de cualquier HTML.

Guardar y modificar listas en la sesión

Un array guardado en la sesión se manipula como cualquier otro. Para añadir un elemento basta con `[]`. Para modificar o borrar, se trabaja sobre `$_SESSION["clave"]` directamente.

Ejemplo 1 — Añadir elementos a una lista:

```
// controllers/anadir.php
session_start();

$_SESSION["tareas"] = $_SESSION["tareas"] ?? [];

$_SESSION["tareas"][] = [
    "titulo"     => trim($_POST["titulo"] ?? ""),
    "completada" => false,
];

header("Location: ../index.php");
exit();
```

Ejemplo 2 — Borrar y modificar un elemento por su posición:

```
session_start();
$pos = (int) ($_POST["pos"] ?? -1);

if (isset($_SESSION["tareas"][$pos])) {
    // Marcar como completada
    $_SESSION["tareas"][$pos]["completada"] = true;

    // O borrarla (y reindexar para que no queden huecos)
    unset($_SESSION["tareas"][$pos]);
    $_SESSION["tareas"] = array_values($_SESSION["tareas"]);
}
```

Atención: Tras un `unset()` en un array indexado, las posiciones quedan con «huecos» (0, 2, 3…). `array_values()` las vuelve a numerar desde 0.

Mensajes flash

Un **mensaje flash** es un aviso que se muestra **una sola vez**: el controller lo guarda en la sesión, redirige, y la página lo muestra y lo borra inmediatamente para que no vuelva a aparecer al recargar.

Ejemplo 1 — Guardar el mensaje en el controller:

```
// controllers/guardar.php
session_start();

$_SESSION["flash"] = [
    "tipo"  => "exito",                    // "exito" o "error"
    "texto" => "Operación realizada correctamente",
];

header("Location: ../index.php");
exit();
```

Ejemplo 2 — Mostrarlo y borrarlo en la página:

```
<?php
session_start();

$flash = $_SESSION["flash"] ?? null;
unset($_SESSION["flash"]);                // se borra para que no se repita
?>
<?php if ($flash !== null): ?>
    <div class="aviso <?= $flash["tipo"] ?>">
        <?= htmlspecialchars($flash["texto"]) ?>
    </div>
<?php endif; ?>
```

Consejo: Importante: leer el mensaje **y** borrarlo en el mismo sitio, antes de pintar el HTML.

Proteger páginas y cerrar sesión

Para que una página solo sea accesible en cierto estado (por ejemplo, haber iniciado sesión), se comprueba al principio y se redirige si no se cumple. Este bloque suele ponerse en un archivo que se incluye en las páginas protegidas.

Ejemplo 1 — Comprobación al principio de la página:

```
<?php
session_start();

if (!isset($_SESSION["usuario"])) {
    header("Location: login.php");
    exit();
}
// Si llegamos aquí, hay usuario en sesión
?>
<h1>Hola, <?= htmlspecialchars($_SESSION["usuario"]) ?></h1>
```

Ejemplo 2 — Cerrar sesión:

```
<?php
// controllers/logout.php
session_start();
session_destroy();         // elimina todos los datos de la sesión

header("Location: ../login.php");
exit();
```

Ejemplo 3 — Reiniciar solo una parte de la sesión:

```
session_start();

// Volver al estado inicial SIN cerrar la sesión completa
unset($_SESSION["tareas"]);
$_SESSION["tareas"] = [];
```

9. Programación orientada a objetos (POO)

Clases y objetos

Una **clase** es una plantilla que define qué datos (**propiedades**) y qué acciones (**métodos**) tiene un tipo de cosa. Un **objeto** es una copia concreta creada a partir de esa plantilla con `new`. Dentro de la clase, `$this` se refiere al objeto actual.

Ejemplo 1 — La clase más sencilla:

```
class Mascota {
    public string $nombre;
    public int $edad;

    public function __construct(string $nombre, int $edad) {
        $this->nombre = $nombre;
        $this->edad   = $edad;
    }

    public function presentarse(): string {
        return "Soy " . $this->nombre . " y tengo " . $this->edad . " años";
    }
}

$m = new Mascota("Luna", 3);
echo $m->presentarse();        // Soy Luna y tengo 3 años
echo $m->nombre;               // Luna
```

Ejemplo 2 — Varios objetos de la misma clase:

```
$mascotas = [
    new Mascota("Luna", 3),
    new Mascota("Toby", 7),
];

foreach ($mascotas as $m) {
    echo $m->presentarse() . "<br>";
}
```

Consejo: `->` sirve para acceder a propiedades y métodos de un objeto (como el punto en JavaScript). El constructor `__construct` se ejecuta automáticamente al hacer `new`.

Visibilidad, getters y setters

Modificador	Se puede usar...
`public`	Desde cualquier sitio
`private`	Solo dentro de la propia clase
`protected`	Dentro de la clase y de sus clases hijas

Lo habitual es poner las propiedades como `private` y dar acceso mediante métodos: un **getter** para leer y un **setter** para modificar. Así el setter puede **validar** antes de aceptar un valor.

Ejemplo 1 — Propiedad privada con getter y setter que valida:

```
class Cuenta {
    private float $saldo = 0;

    public function getSaldo(): float {
        return $this->saldo;
    }

    public function ingresar(float $cantidad): bool {
        if ($cantidad <= 0) {
            return false;               // rechazado: no se modifica nada
        }
        $this->saldo += $cantidad;
        return true;
    }
}

$c = new Cuenta();
$c->ingresar(50);
echo $c->getSaldo();       // 50
// echo $c->saldo;         // Error: la propiedad es privada
```

Ejemplo 2 — Un método que decide según el estado del objeto:

```
class Pelicula {
    private bool $vista = false;

    public function __construct(private string $titulo) {}      // propiedad
declarada en el constructor

    public function marcarVista(): void {
        $this->vista = true;
    }

    public function getEstado(): string {
        return $this->vista ? "Vista" : "Pendiente";
    }

    public function getTitulo(): string {
        return $this->titulo;
    }
}

$p = new Pelicula("Alien");
$p->marcarVista();
echo $p->getTitulo() . ": " . $p->getEstado();     // Alien: Vista
```

Atención: Si declaras el parámetro del constructor con visibilidad (`private string $titulo`), PHP crea la propiedad y le asigna el valor automáticamente. Es una forma abreviada de escribir lo mismo.

Constantes y atributos de la clase

Una constante de clase se declara con `const` y se lee con `NombreClase::NOMBRE` (o `self::NOMBRE` dentro de la clase). Es útil para valores fijos que pertenecen a la clase.

Ejemplo 1 — Constante de clase:

```
class Tarifa {
    const IVA = 21;

    public static function conIva(float $precio): float {
        return $precio * (1 + self::IVA / 100);
    }
}

echo Tarifa::IVA;                  // 21
echo Tarifa::conIva(100);          // 121
```

Ejemplo 2 — Usar constantes globales (define) dentro de un método:

```
// config.php
define("LIMITE", 200);

// En la clase, las constantes globales se leen directamente
class Cuenta {
    private float $saldo = 0;

    public function puedeRetirar(float $cantidad): bool {
        return $this->saldo - $cantidad >= -LIMITE;
    }
}
```

Herencia

Una clase puede **heredar** de otra con `extends`: recibe sus propiedades y métodos y puede añadir los suyos o **sobrescribirlos**. `parent::` llama al método de la clase padre.

Ejemplo 1 — Clase hija que añade una propiedad:

```
class Animal {
    public function __construct(protected string $nombre) {}

    public function sonido(): string {
        return "...";
    }

    public function presentarse(): string {
        return $this->nombre . " dice " . $this->sonido();
    }
}

class Perro extends Animal {
    public function sonido(): string {          // sobrescribe el del padre
        return "guau";
    }
}

$p = new Perro("Toby");
echo $p->presentarse();          // Toby dice guau
```

Ejemplo 2 — Usar parent:: para ampliar el comportamiento:

```
class Gato extends Animal {
    public function __construct(string $nombre, private int $vidas = 7) {
        parent::__construct($nombre);              // el constructor del padre
guarda el nombre
    }

    public function presentarse(): string {
        return parent::presentarse() . " (" . $this->vidas . " vidas)";
    }
}

echo (new Gato("Mishi"))->presentarse();           // Mishi dice ... (7 vidas)
```

Una clase que gestiona una lista de objetos

Es un patrón muy habitual: una clase «colección» guarda un array de objetos y ofrece métodos para añadir, buscar, contar o calcular totales. Con ello el resto del programa no manipula el array directamente.

Ejemplo 1 — Colección con añadir, buscar y total:

```
class Tarea {
    public function __construct(
        public int $id,
        public string $titulo,
        public bool $hecha = false
    ) {}
}

class ListaTareas {
    private array $tareas = [];

    public function anadir(Tarea $t): void {
        $this->tareas[] = $t;
    }

    public function buscar(int $id): ?Tarea {          // ?Tarea = devuelve una
Tarea o null
        foreach ($this->tareas as $t) {
            if ($t->id === $id) {
                return $t;
            }
        }
        return null;
    }

    public function contarPendientes(): int {
        $n = 0;
        foreach ($this->tareas as $t) {
            if (!$t->hecha) {
                $n++;
            }
        }
        return $n;
    }

    public function todas(): array {
        return $this->tareas;
    }
}
```

Ejemplo 2 — Usarla:

```
$lista = new ListaTareas();
$lista->anadir(new Tarea(1, "Comprar pan"));
$lista->anadir(new Tarea(2, "Estudiar PHP"));

$t = $lista->buscar(2);
if ($t !== null) {
    $t->hecha = true;
}

echo $lista->contarPendientes();      // 1
```

Guardar objetos en la sesión

Los objetos se pueden guardar en `$_SESSION`: PHP los convierte a texto al terminar la petición y los reconstruye en la siguiente. Para que esa reconstrucción funcione, **la clase debe estar cargada antes de `session_start()`**; si no, el objeto aparecerá como «__PHP_Incomplete_Class».

Ejemplo 1 — Orden correcto: primero la clase, después la sesión:

```
<?php
require "classes/ListaTareas.php";       // 1.º cargar las clases
session_start();                         // 2.º abrir la sesión

if (!isset($_SESSION["lista"])) {
    $_SESSION["lista"] = new ListaTareas();
}
$lista = $_SESSION["lista"];             // el mismo objeto de la petición
anterior
```

Ejemplo 2 — Modificar el objeto en un controller:

```
<?php
// controllers/anadir.php
require "../classes/ListaTareas.php";
session_start();

$lista = $_SESSION["lista"] ?? new ListaTareas();
$lista->anadir(new Tarea(3, trim($_POST["titulo"])));
$_SESSION["lista"] = $lista;             // se vuelve a guardar

header("Location: ../index.php");
exit();
```

Atención: Cada archivo que use el objeto de la sesión debe cargar la clase **antes** de `session_start()`, incluidos los controllers. Es el error más típico al mezclar POO y sesiones.

Consejo: Una clase por archivo, con el nombre de la clase como nombre de archivo (`classes/Tarea.php`), y cárgalas con `require_once`.

10. Errores habituales y cómo depurar

Depurar: ver qué contiene una variable

Cuando algo no funciona, lo primero es **mirar qué valor tiene realmente la variable**.

Ejemplo 1 — var_dump y print_r:

```
$datos = ["Ana", 25, true];

var_dump($datos);          // muestra tipos y valores (útil para ver int vs
string, true vs "1")
print_r($datos);           // más compacto: solo valores

echo "<pre>";
print_r($_POST);           // <pre> conserva el formato en el navegador
echo "</pre>";
```

Ejemplo 2 — Comprobar si se está ejecutando una parte del código:

```
echo "Llego aquí";         // pista rápida
var_dump($_SESSION);       // ¿qué hay en la sesión?
exit();                    // detiene el script para ver solo la salida de
depuración
```

Consejo: Cuando termines, elimina los `var_dump`, `print_r` y `exit()` de depuración.

Mensajes de error frecuentes

Mensaje	Causa más probable	Solución
`Parse error: syntax error, unexpected ...`	Falta un `;`, una llave, un paréntesis o una comilla	Revisa la línea indicada **y la anterior**
`Undefined variable $x`	La variable no existe: nombre mal escrito o está fuera de su ámbito (función)	Revisa el nombre; pásala como parámetro
`Undefined array key "x"`	La clave no existe en el array	Comprueba con `isset()` o usa `??`
`Trying to access array offset on value of type null`	Usas `$x["a"]` pero `$x` es `null`	Comprueba que `$x !== null` antes
`Call to undefined function`	Función mal escrita o no cargada	Revisa el nombre y el `require`
`Cannot redeclare function`	La función se ha cargado dos veces	Usa `require_once` o no la definas dos veces
`Failed opening required`	Ruta incorrecta	Revisa la ruta; en subcarpetas usa `../`
`headers already sent`	Hubo salida (HTML, echo, espacios) antes de `header()` o `session_start()`	Mueve la lógica arriba, antes del HTML
`Division by zero`	Divides entre 0	Comprueba el divisor antes
`Unsupported operand types`	Operas con un array o `null` como si fuera número	Comprueba qué contiene la variable con `var_dump`

Errores lógicos típicos

Síntoma	Causa habitual
La condición siempre sale verdadera	Usaste `=` en lugar de `==` o `===`
La media o el total salen mal	Dividir antes de terminar de sumar, o no inicializar el acumulador a `0`
El máximo o mínimo sale mal	Empezar con un valor inicial incorrecto; se debe empezar con el primer elemento
Se muestra `1` o nada en vez de Sí/No	Imprimir un booleano directamente en vez de usar un ternario
El checkbox no «llega» al desmarcarlo	Un checkbox desmarcado no se envía: usa `isset()`
Se duplica una operación al recargar	Procesar un POST sin redirigir después: usa `header()` + `exit()`
Los datos no se guardan entre páginas	Falta `session_start()` o está después del HTML
Un número aparece como `10.5` en vez de `10,50`	Falta `number_format()` al mostrarlo
`"10" < "9"` sale verdadero	Se comparan como texto; convierte con `(int)` o `(float)`
Al borrar de un array quedan huecos	Falta `array_values()` tras el `unset()`

Antes de entregar: lista de comprobación

- Todas las páginas cargan **sin ningún aviso ni error** de PHP en pantalla.
- Has probado **todos los casos** que indica el enunciado, incluidos los casos límite (lista vacía, valor 0, campo vacío).
- Los textos que escribe el usuario se muestran con `htmlspecialchars()`.
- Las operaciones con formularios terminan con `header()` + `exit()`.
- Los importes se muestran con `number_format()` y el formato pedido.
- La estructura de carpetas y los nombres de archivo son los que pide el enunciado.
- No quedan `var_dump`, `print_r` ni código comentado de pruebas.

Chuleta final: referencia rápida

Sintaxis básica

Quiero…	Escribo…
Mostrar un valor en HTML	`<?= $x ?>`
Valor por defecto si no existe	`$x ?? "defecto"`
Condición en una línea	`$a > 5 ? "sí" : "no"`
Recorrer un array con claves	`foreach ($arr as $clave => $valor)`
Añadir al final de un array	`$arr[] = $valor;`
Contar elementos	`count($arr)`
Comprobar si existe una clave	`isset($arr["clave"])`
Comprobar si un valor está en el array	`in_array($v, $arr, true)`
Ordenar con criterio propio	`usort($arr, fn($a, $b) => $a["x"] <=> $b["x"]);`
Dos decimales y separadores	`number_format($n, 2, ",", ".")`
Texto seguro para HTML	`htmlspecialchars($texto)`

Formularios y sesiones

Quiero…	Escribo…
Leer un campo POST	`trim($_POST["campo"] ?? "")`
Saber si un checkbox está marcado	`isset($_POST["campo"])`
Saber si se envió el formulario	`$_SERVER["REQUEST_METHOD"] === "POST"`
Redirigir	`header("Location: x.php"); exit();`
Abrir la sesión	`session_start();` (primera línea)
Guardar en la sesión	`$_SESSION["clave"] = $valor;`
Borrar un dato de la sesión	`unset($_SESSION["clave"]);`
Cargar un archivo imprescindible	`require "archivo.php";`

Orden recomendado dentro de un archivo

```php
<?php
// 1. session_start() (si se usa)
// 2. require de clases / config / datos / funciones
// 3. Recoger datos de GET / POST / sesión
// 4. Validar y calcular
// 5. Redirigir si hace falta (header + exit)
?>
<!-- 6. HTML: solo mostrar lo ya calculado -->
```